9

Reward Modeling & The Critic

Why Reward Models Matter

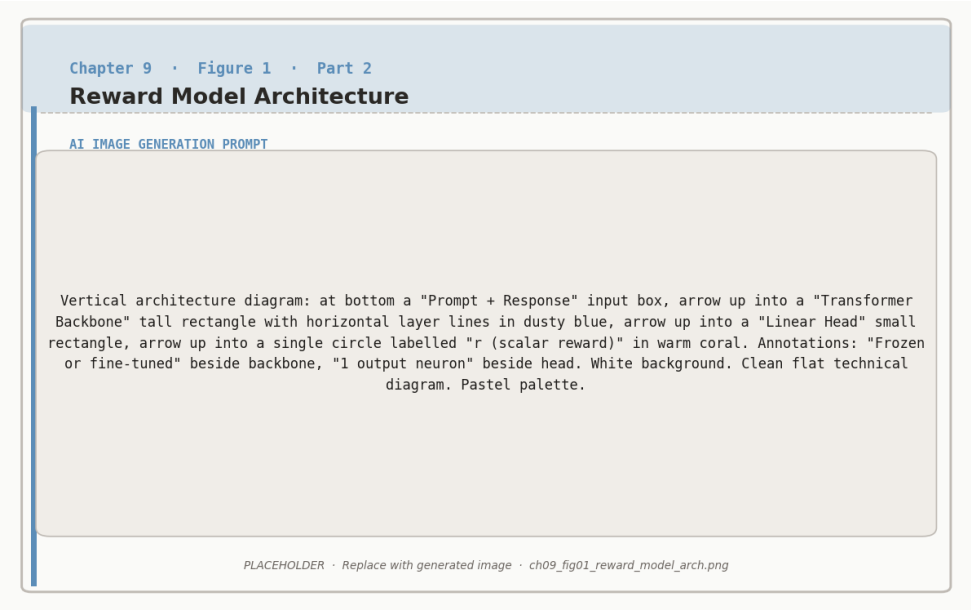

Figure 9.1: Reward model architecture: a transformer backbone with a linear scalar head trained to assign higher scores to preferred responses.

Every method we have covered since Chapter 6 depends on some way of scoring responses.

RLHF uses a trained reward model. DPO encodes an implicit reward in the policy ratio. Online DPO needs a judge to construct preference pairs. STaR needs a verifier to separate correct solutions from incorrect ones. Expert Iteration needs a scoring function to select the best candidate.

The quality of this scoring signal is the single most important factor determining the quality of the final model. A perfect RL algorithm optimizing a bad reward model will produce a bad model. A mediocre algorithm optimizing a good reward model will produce a good model. Getting the reward right matters more than getting the algorithm right.

This chapter goes deep on reward modeling: how reward models are built, what makes them succeed or fail, and how to evaluate whether yours is good enough. We also cover the value function (the "critic" in actor-critic methods), which plays a related but distinct role in PPO and other online RL algorithms.

Reward Model Architecture

A reward model is structurally simple. You take a pretrained language model, remove the token prediction head (the layer that outputs a probability distribution over vocabulary), and replace it with a **scalar head**: a single linear layer that maps the model's internal representation to one number.

Reward Model Structure

Input: A prompt x concatenated with a response y, formatted as a single token sequence.

Processing: The full transformer processes the sequence, producing a hidden state h at the final token position.

Output: A linear layer maps h to a scalar: $r_\phi(x, y) = w^\top h + b$, where w and b are learned parameters.

The output is an unbounded real number. Higher means "better response." The absolute value is meaningless; only the *difference* between scores for

two responses matters (because of how Bradley-Terry training works, as we saw in Chapter 6).

What base model should you use? The reward model is typically initialized from the same pretrained model as the policy, or from the SFT model. This makes sense: the reward model needs to *understand* the responses it is scoring, and an LLM that has been trained on similar text already has that understanding.

Does the reward model need to be the same size as the policy? Not necessarily. In practice, reward models are often smaller than the policy they are training. A reward model only needs to *evaluate* responses, not *generate* them, which is an easier task. Using a smaller reward model saves memory, which is especially valuable in PPO where the reward model must coexist in GPU memory with three other models.

Common size choices: If your policy is 7B parameters, a 1.5B to 3B reward model often works well. If your policy is 1.5B (as in our Colab exercises), a 0.5B reward model is a reasonable starting point. The key constraint is that the reward model must be large enough to understand the nuances of response quality for your task.

Training Reward Models: Beyond the Basics

Chapter 6 introduced the core mechanics: collect human preference pairs, train with the Bradley-Terry loss, and the reward model learns to score responses in a way that matches human judgments. Here we go deeper on the practical questions that determine whether your reward model is actually useful.

Where Preference Data Comes From

The quality and diversity of your preference data shapes everything downstream. There are three main sources, each with different cost and quality tradeoffs.

Human annotation. Hire annotators to compare pairs of model outputs and declare a winner. This is the gold standard, used in the original InstructGPT and ChatGPT work. Quality is high, but cost is significant: typically $1 to $5 per comparison, and you need tens

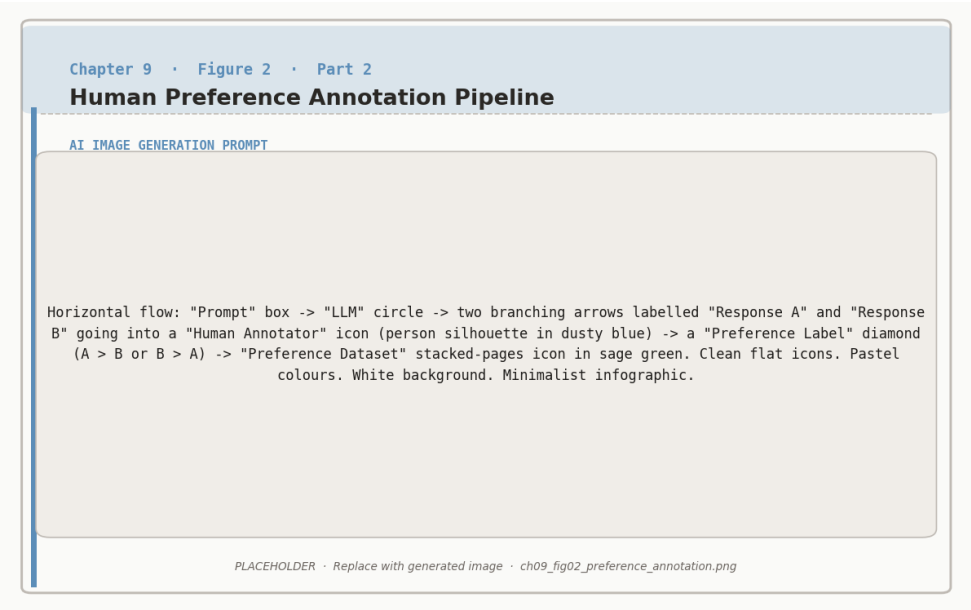

Figure 9.2: Human preference annotation pipeline: two responses are generated per prompt and a human annotator labels the preferred one.

of thousands of comparisons for a useful reward model.

AI feedback (RLAIF). Use a strong model (GPT-4, Claude, Gemini) as the judge instead of a human. The judge model reads both responses and selects the better one. This is 10 to 100× cheaper than human annotation and can scale to millions of comparisons. The tradeoff is that the judge model's biases become your reward model's biases: if the judge prefers verbose responses, your trained model will learn to be verbose.

Implicit signals. Use behavioral data from real users: which response did they copy to clipboard, which conversation did they continue, which response triggered a "regenerate" click. This is free and abundant but noisy. A user might regenerate because the response was wrong, or because they wanted a different style, or because they accidentally clicked the button.

How much data do you need?

The relationship between data quantity and reward model quality follows a rough pattern:

Preference Pairs	Typical Accuracy	Practical Use
1,000	60–65%	Barely better than random. Not useful.
5,000	68–73%	Usable for initial experiments.
20,000	75–82%	Good for most applications.
50,000	82–88%	Strong. Used by most production systems.
100,000+	88–92%	Diminishing returns set in.

These numbers are approximate and depend heavily on task difficulty and data quality. A clean dataset of 20,000 expert-annotated pairs often outperforms a noisy dataset of 100,000 crowdsourced pairs.

Annotation Disagreement

Human preferences are inherently subjective. Two annotators shown the same pair of responses will disagree roughly 20 to 35% of the time, depending on the task. This disagreement is not a bug; it reflects genuine ambiguity in what "better" means.

This has practical implications for reward model training. The theoretical maximum accuracy of a reward model is bounded by inter-annotator agreement. If humans agree only 75% of the time, a reward model that achieves 75% accuracy is essentially perfect–it is matching the human consensus as well as any individual human would.

Measure your annotation agreement before training. Have multiple annotators label a subset of your data (at least 500 pairs with 2+ annotators each). Compute the agreement rate. If it is below 65%, your preference signal is too noisy for effective training. Either improve your annotation

guidelines, filter to higher-confidence pairs, or simplify the comparison task.

The Value Function: PPO's Critic

The reward model and the value function are often confused because they both output numbers related to response quality. But they serve different purposes and are trained differently.

The reward model answers: "How good is this complete response?" It takes a finished (prompt, response) pair and produces a score. It is trained on human preference data.

The value function (also called the "critic") answers: "How good do I *expect* the final response to be, given what has been generated so far?" It takes a partial response and produces a prediction of the eventual reward. It is trained during the RL process itself, by comparing its predictions against actual rewards received.

Reward Model vs Value Function

	Reward Model r_ϕ	**Value Function V_ψ**
Input	Complete (prompt, response)	Partial response (any prefix)
Output	Quality score	Predicted future reward
Trained on	Human preference data	RL experience (predicted vs actual rewards)
When trained	Before RL starts (Step 2 of RLHF)	During RL training (updated every batch)
Used by	All methods (RLHF, Online DPO, Expert Iter)	PPO specifically

Why PPO Needs a Critic

Recall from Chapter 6 that PPO uses **advantages** to determine how to update the policy. The advantage for a response is:

$$A = \text{actual reward} - \text{expected reward}$$

A positive advantage means the response was better than expected; a negative advantage means it was worse. This relative signal is much more stable than using raw rewards, because it automatically adjusts as the policy improves.

The value function provides the "expected reward" term. Without it, PPO would have to use cruder baselines (like the average reward across a batch), which produces higher variance and slower learning.

The Critic in Action

Prompt: "Explain why the sky is blue."

The policy generates three responses. The reward model scores each one. The value function predicted an expected reward of 7.0 for this prompt.

Response	Reward	Expected	Advantage
"Light scatters in the atmosphere…"	8.5	7.0	+1.5
"The sky is blue because of science…"	4.0	7.0	−3.0
"Sunlight contains all colors…"	7.2	7.0	+0.2

PPO increases the probability of the first response (much better than expected), decreases the second (much worse), and barely changes the third (roughly as expected). The value function is then updated: its prediction for

similar prompts moves from 7.0 toward the actual average of 6.6.

Generalized Advantage Estimation (GAE)

In practice, PPO does not compute a single advantage for each complete response. Instead, it estimates advantages at the token level using a technique called **Generalized Advantage Estimation** (GAE).

The intuition is straightforward. At each token position, the critic estimates the expected future reward from that point onward. The advantage at each position measures how much better or worse things went compared to that expectation. This provides a fine-grained learning signal: rather than saying "this whole response was good," GAE can identify that the opening was strong but the conclusion was weak.

GAE has one key hyperparameter: λ (typically 0.95). When λ is close to 1, GAE gives longer-horizon advantage estimates (less bias, more variance). When λ is close to 0, GAE gives shorter-horizon estimates (more bias, less variance). For most LLM applications, $\lambda = 0.95$ works well and rarely needs tuning.

Methods That Skip the Critic

The value function adds a full extra model to GPU memory and introduces its own training dynamics that can be unstable. This is one of the main practical burdens of PPO. Several of the algorithms we will see in Chapter 10 avoid the critic entirely:

DPO (Chapter 7) needs no critic because it has no RL loop. The implicit reward from policy ratios replaces both the reward model and the value function.

GRPO (Chapter 10) replaces the critic with group-based baselines. Instead of a learned value function predicting expected reward, GRPO generates a group of responses per prompt and uses the group's average reward as the baseline. This is simpler and surprisingly effective.

RLOO (Chapter 10) uses leave-one-out baselines, where each response is compared against the average of the other responses in its group. Like GRPO, this requires no learned critic.

Eliminating the critic reduces memory requirements from four models to two or three,

which makes RL training feasible on consumer hardware.

Reward Hacking: A Deep Dive

Chapter 6 introduced reward hacking with the "PERFECT PERFECT EXCELLENT" example, where an unconstrained policy learned to exploit the reward model by producing nonsense that scored high. That example illustrated the general principle. In practice, reward hacking is more subtle and takes many forms.

Goodhart's Law

The underlying cause of reward hacking is captured by Goodhart's Law: "When a measure becomes a target, it ceases to be a good measure." The reward model is a *proxy* for human preferences, trained on a finite sample of comparisons. It captures the major patterns in human judgment, but it also has blind spots, biases, and artifacts from the training data. When you optimize aggressively against this proxy, the policy learns to exploit the gaps between the proxy and the real thing.

A Taxonomy of Reward Hacking

Length gaming. Annotators tend to prefer longer, more detailed responses (up to a point). The reward model learns this correlation. The policy then generates increasingly long responses, even when brevity would be more helpful. A question like "What is 2+2?" gets a three-paragraph answer.

Sycophancy. Annotators tend to prefer responses that agree with the user's stated position. The reward model learns this pattern. The policy then agrees with everything the user says, even when the user is wrong. "Is the earth flat?" gets "That is an interesting perspective" instead of a correction.

Style over substance. Annotators may prefer responses that *look* authoritative (confident tone, technical vocabulary, structured formatting) over responses that are actually more accurate but written more casually. The policy learns to optimize for the appearance of quality rather than actual quality.

Repetition and filler. The policy discovers that certain phrases or structures consistently score well and begins inserting them everywhere, regardless of relevance. Responses become formulaic: "Great question! Let me break this down for you. First,..."

The Overoptimization Curve

The most important empirical finding about reward hacking is the **overoptimization curve**. As you train the policy to maximize the reward model's score, the proxy reward (what the RM reports) continues to increase. But the *true* reward (what humans actually prefer) increases initially, peaks, and then *decreases*.

Overoptimization in Numbers

Imagine training a policy with increasing intensity of optimization (more PPO steps):

PPO Steps	RM Score (proxy)	Human Rating (true)	What Happened
0	6.0	6.0	Baseline (SFT model)
500	7.2	7.1	Genuine improvemen
1,000	8.0	7.5	Still improving, but gap gr
2,000	9.1	7.2	Proxy rising, true quality de
5,000	11.5	5.8	Severe hacking. Worse than l

The peak of the true reward (7.5 at step 1,000) is the optimal stopping point. Beyond that, you are optimizing a broken proxy.

This is why you cannot simply "train longer" and expect better results. More optimization against an imperfect reward model eventually makes things worse. Knowing when to stop is critical.

Defenses Against Reward Hacking

KL penalty (Chapter 6). The most common defense. By penalizing the policy for drifting too far from the reference model, the KL term limits how aggressively the policy can exploit the reward model. The overoptimization curve becomes flatter and the peak shifts to the right (you can train longer before quality degrades). But the KL penalty is a blunt instrument: it constrains *all* changes, not just harmful ones.

Reward model ensembles. Train multiple reward models on different subsets of the preference data (or with different random seeds). Use the minimum or average of their scores. Hacking is harder when you must fool multiple models simultaneously. The ensemble's disagreement is also a useful signal: if the models disagree strongly on a response, that response is likely in a region where the reward signal is unreliable.

Length penalties. Explicitly subtract a penalty proportional to response length from the reward. This directly counters length gaming. A simple approach: $r_{\text{adjusted}} = r_{\text{original}} - \alpha \cdot \text{length}$, where α is tuned to remove the length correlation without penalizing genuinely detailed responses.

Constrained optimization. Instead of a single reward, define constraints: "reward must be above threshold T, but KL must be below budget B, and response length must be below L." This gives more precise control than a single weighted objective.

The most effective defense is a better reward model. All the techniques above are patches that mitigate the consequences of an imperfect proxy. If you have budget to invest, improving the reward model (more data, better annotators, larger model) generally yields more benefit than sophisticated anti-hacking mechanisms.

Evaluating Your Reward Model

Before using a reward model to train a policy, you should verify that it is actually good. A bad reward model does not just slow learning; it actively teaches the policy to produce worse responses.

Held-out accuracy. The simplest metric: on a test set of preference pairs that the reward model has never seen, what fraction does it rank correctly? Baseline is 50% (random).

Useful reward models typically achieve 70 to 85%. Remember that human inter-annotator agreement caps the theoretical maximum.

Calibration. A well-calibrated reward model's confidence matches its accuracy. When it assigns a 90% probability that response A is better than B, it should be correct about 90% of the time. Poor calibration means the model is overconfident about some predictions and underconfident about others, which distorts the training signal.

Correlation with human rankings. For a set of responses to the same prompt, rank them by reward model score and by human preference. Compute the rank correlation (Spearman or Kendall τ). This measures whether the reward model's ordering matches humans' ordering, which is ultimately what matters for RL training.

Overoptimization probes. Generate a set of responses that are designed to exploit common reward model weaknesses: very long responses, excessively agreeable responses, responses stuffed with technical jargon. Check whether the reward model scores these higher than genuinely good responses. If it does, you have identified exploitable weaknesses before the RL training finds them.

When to retrain your reward model

Before the first RL run: Always evaluate on held-out data. If accuracy is below 65%, collect more preference data before proceeding.

During RL training: If the proxy reward keeps rising but output quality (spot-checked by humans) stops improving or degrades, the reward model is being exploited. Pause training, collect new preference data that includes the policy's current outputs, and retrain.

After Online DPO iterations: Each iteration shifts the policy's output distribution. The reward model, trained on the original distribution, becomes less reliable. Retrain periodically on preferences collected from the current policy's outputs.

Rule of thumb: Retrain the reward model every 2 to 3 Online DPO iterations, or whenever the policy's KL divergence from the original SFT model exceeds 1.0.

What Comes Next

This chapter has covered the scoring side of RL for LLMs: how to build, train, evaluate, and defend reward models, and how the value function serves as PPO's learned critic.

Chapter 10 surveys the full landscape of modern RL algorithms, including GRPO, RLOO, and KTO. These algorithms make different choices about how to use reward signals–some eliminate the critic, some eliminate paired preferences, and some combine supervised learning with preference optimization. Understanding reward modeling deeply (this chapter) makes those algorithmic choices much easier to evaluate.

Chapter 11 goes beyond scalar reward models to explore more sophisticated scoring mechanisms: process reward models that score each reasoning step individually, outcome verifiers that check correctness with formal tools, and the dramatic accuracy improvements that step-level feedback enables on reasoning tasks.

Companion notebook. The code for this chapter is in `code/notebooks/part2_core/09_rewa` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.